3.1 C#核心_封装

主要就是学面向对象这块

- 三大特性
  - 封装
  - 继承
  - 多态
- 七大原则
  - 核心就是"高内聚、低耦合"
  - 常见就是 单一职责、开闭原则、依赖倒置
- 其他有的没的

各种数据类型的默认值

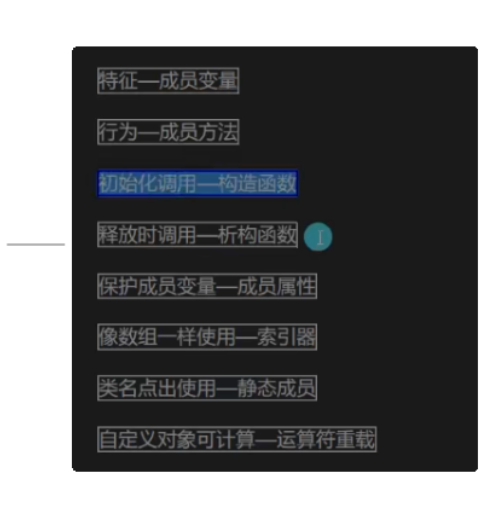
类型类别	具体类型	默认值	备注
整数类型	int	0	short/long/byte/sbyte 同理，都是 0
浮点类型	float	0.0f	double 是 0.0、decimal 是 0.0m
布尔类型	bool	false	只有 true/false 两种值
字符类型	char	'\0'（空字符）	对应 ASCII 码的 0
字符串类型	string	null	引用类型，默认无指向
枚举类型	自定义枚举（如 E_sexType）	枚举的第一个值	比如 E_sexType 默认是 male（如果 male 是第一个）
结构体类型（值类型）	自定义 struct（如 position）	所有成员取默认值	比如 position 的 x/y/z 都是 0
类（引用类型）	自定义 class（如 Student）	null	栈区变量无指向，堆区无对象
数组	任意数组（如 int []）	null	引用类型，默认无指向
可空类型	int?/bool?（如 int? num）	null	专门用来存 "无值" 的类型

- 直接Console.writeLine(default(int))就可以查看其默认类型 —— char和Person这种默认null打印出来啥都看不见

类

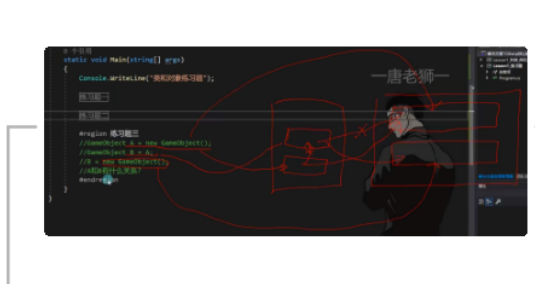
注意事项

- 同一个namespace中的class类不能重名
- 帕斯卡命名法
- 声明在命名空间namespace中
  - 枚举和函数声明在类class中 —— 当然,类class中也可以声明类,叫做 内部类
  - 结构体和类声明在命名空间namespace中
- 和结构体区别
  - 反正就是class能定义行为(不仅仅记录信息)
- 类中各种语法的作用

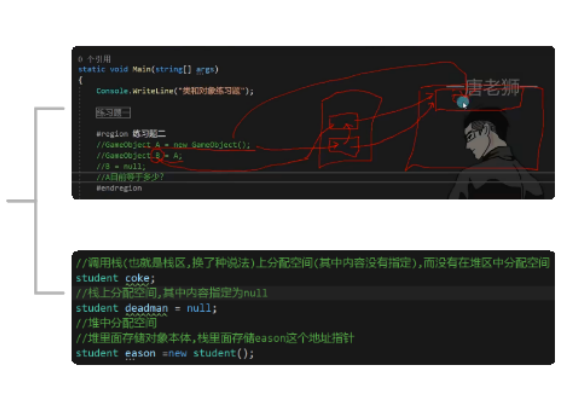


特征—成员变量
行为—成员方法
初始化调用—构造函数
释放时调用—析构函数
保护成员变量—成员属性
像数组一样使用—索引器
类名点出使用—静态成员
自定义对象可计算—运算符重载

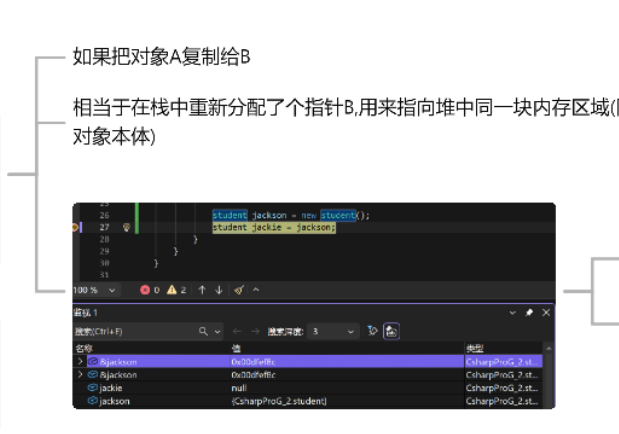
实例化



- 给B类针又找了个新的指向,堆中开辟另一片内存
- 堆区栈区的内存分配



  - 如果把对象A复制给B
  - 相当于在栈中重新分配了个指针B,用来指向堆中同一块内存区域(同一个对象本体)

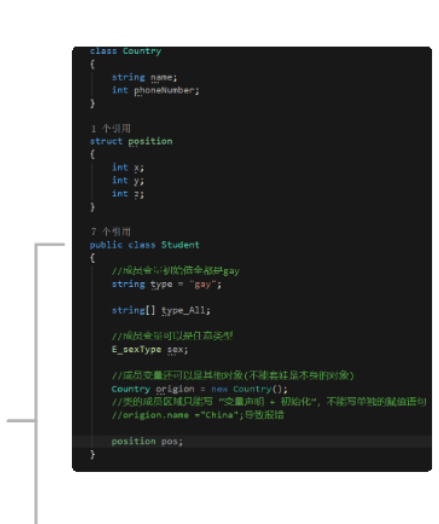


  - 这里断点停在了student jackie = jackson; 还未执行，所以 jackie 这个变量仍然是它的默认值 null
  - jackson和jackie指向同一片内存地址

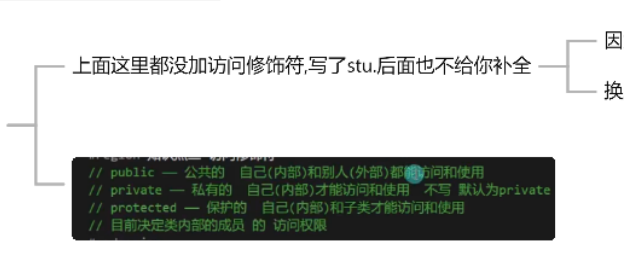
- 实例时的注意事项 —— 注意类class对象是引用类型 —— 引用类型使用前必须赋值(初始值)
  - 输入文字
    - 仅仅Student stu之后直接使用stu.name会报错 —— 需要Student stu = new student();
    - 仅仅int[] arr;之后直接使用arr[0]变量会报错 —— 需要 arr = new int[3]; —— 当然也可以用简便方法赋予初始值
    - 仅仅string name;之后直接使用name变量会报错
      - 字符串不需要new,只用赋予初始值
      - 实际上还是string s = new string('h','e','l','l','o');编译器帮忙简化了而已
  - 赋值null就不会报错,但并没有在堆区分配内存,只是特殊标记 —— Student stu = null;也是可以的

成员变量和成员函数

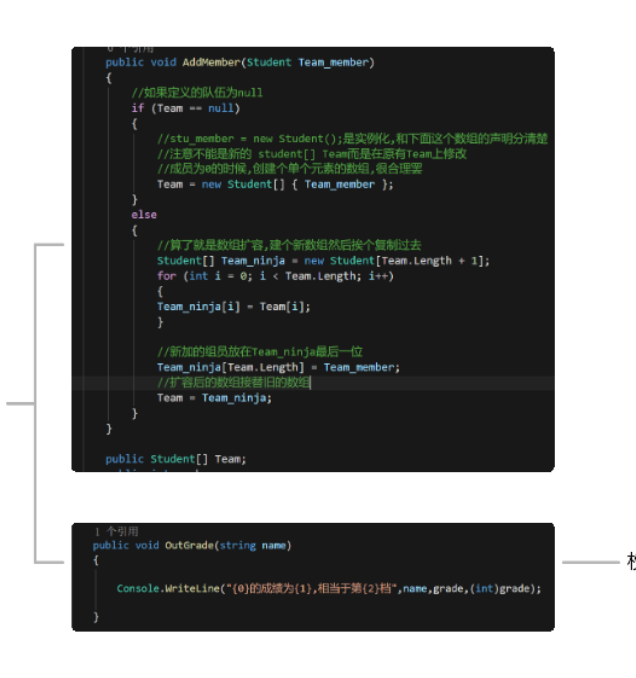
- 成员变量

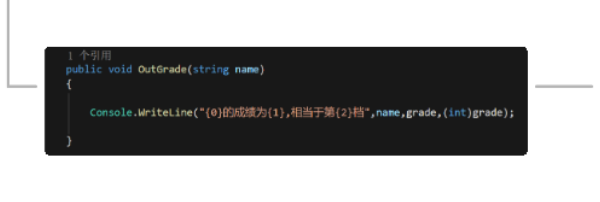


  - 总之就是可以声明的类型任意
  - 注意:如果你在Student类中想要给Person.name赋值,不能直接写
    - Person成员变量里面
    - Student stu;可以
    - Student stu2 = new Student();可以 —— 但是注意如果是同名类 —— 也就是Person成员变量里面Person bro;可以；Person bro =new Person();不可以 —— 导致无限套娃,内存溢出
    - Student stu3 = new Student() { type = "student", age = 18 };可以
    - stu.type = "gay";错误
      - 不管是否初始化过stu对象
      - 放在构造函数才可以
      - 不要问为什么C#就是这么规定的,成员类的成员变量要在构造函数中赋值
- 成员变量的使用
  - 上面这里都没设加访问修饰符,写了stu.后面也不给你补全
    - 因为默认private外部没法使用
    - 换成public就行



- 成员方法(成员函数)
  - 类class比较结构体struct还能定义对象的行为
    - 定义属性:成员变量
    - 定义行为:成员方法
  - 注意事项
    - 声明在类语句块中
    - 不要加static关键字
  - 举例

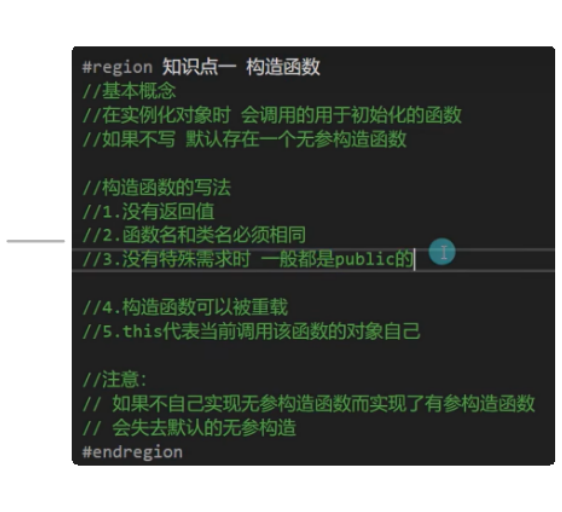




  - 枚举变量声明的时候最好初始化一下

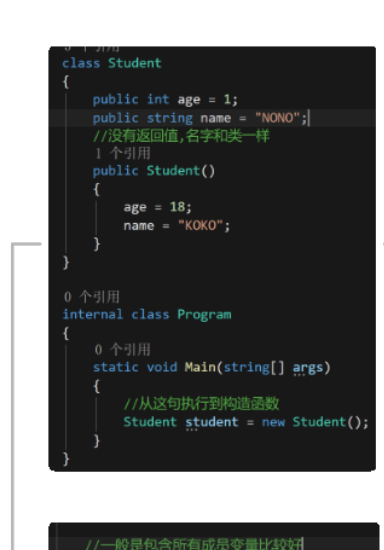
构造函数和析构函数

- 构造函数
  - 注意事项

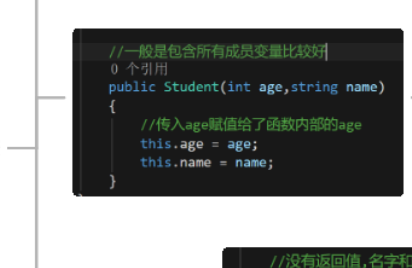


```
#region 知识点一 构造函数
//基本概念
//在实例化对象时 会调用的用于初始化的函数
//如果不写 默认存在一个无参构造函数

//构造函数的写法
//1.没有返回值
//2.函数名和类名必须相同
//3.没有特殊需求时 一般都是public的

//4.构造函数可以被重载
//5.this代表当前调用该函数的对象自己

//注意:
//如果不自己实现无参构造函数而实现了有参构造函数
//会失去默认的无参构造
#endregion
```

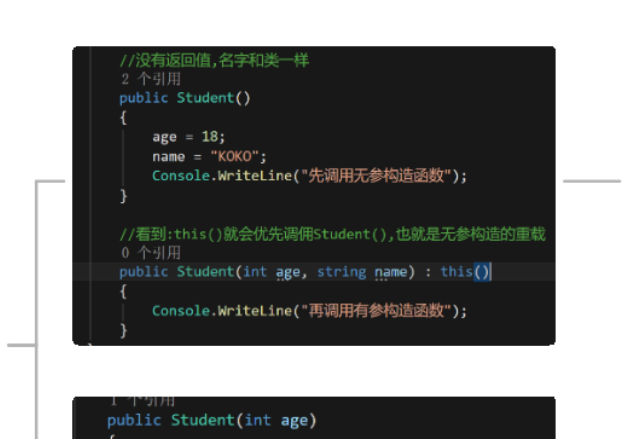
    - 不写也默认存在
    - 实例化的时候,用来初始化
    - 格式自己看前面图



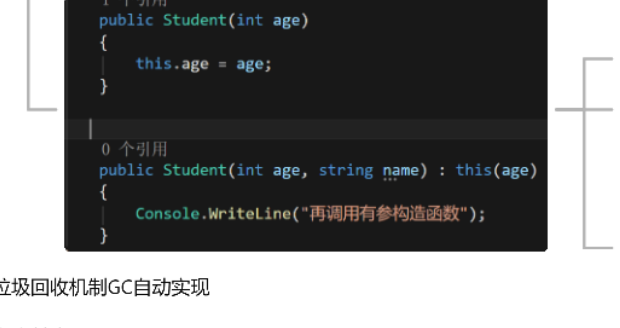
  - 实例化函数的时候,用来初始化对象
    - 直白点就是给对象个初始状态
    - 如果不初始化,age默认为0显然不能导致报错
    - 能予初值虽然也能实现相同的效果,但是构造函数显然更加灵活
  - 如果是无参的构造函数,就是对象不做任何修改 —— 类里面可以有多种构造函数,用重载实现就可以
  - 结构体中不能使用无参的构造函数
    - 现在可以,但是要求很多
    - 结构体值类型,无参构造函数总容易出BUG
  - 构造函数写法



    - 当然也可以通过重载,每次只传入一个参数,分多次写
    - 写了含参构造函数,默认无参构造就没了
  - 特殊写法

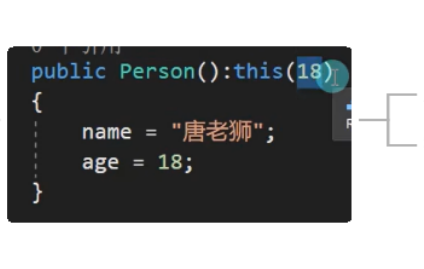


    - 这里先打印无参构造中的内容



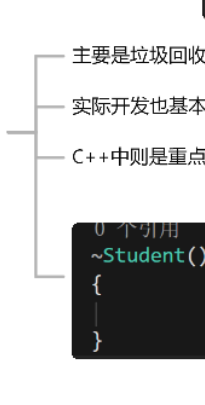
```
public Student(int age)
{
    this.age = age;
}

0 个引用
public Student(int age, string name) : this(age)
{
    Console.WriteLine("再调用有参构造函数");
}
```

    - 这里先执行只有age的重载
    - this()就是个决定执行优先级的玩意儿



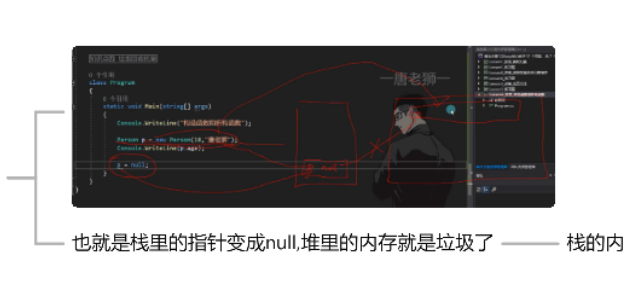
```
public Person():this(18)
{
    name = "唐老狮";
    age = 18;
}
```

    - 直接写个int值也行,默认找符合条件的重载
    - 相当于赋了个初始值18
- 析构函数
  - 居然并不是重点
    - 主要是垃圾回收机制GC自动实现
    - 实际开发也基本用不上
    - C++中则是重点中的重点
  - 随便看看就行

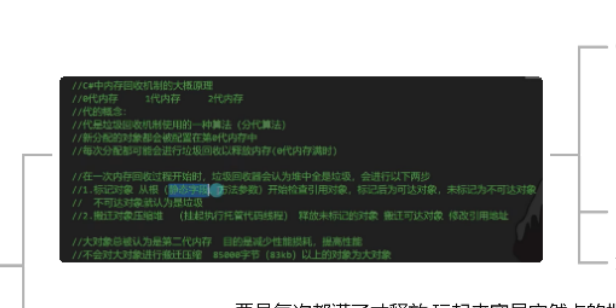


```
0 个引用
~Student()
{

}
```

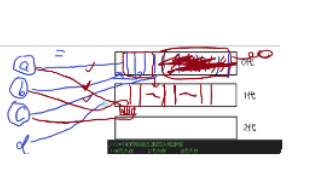
    - 修饰符都用不上
    - 引用类型的堆内存被回收时调用析构函数
- GC垃圾回收机制
  - 垃圾是啥



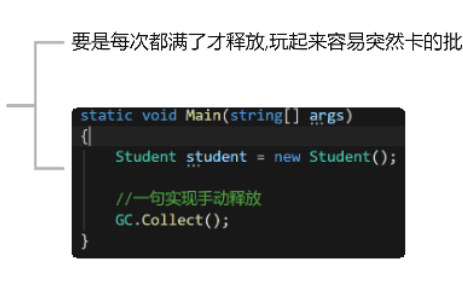
    - 也就是栈里的指针变成null,堆里的内存就是垃圾了 —— 栈的内存是系统自动分配的(有生命周期),不需要手动管理
  - GC原理



    - 0代的回收



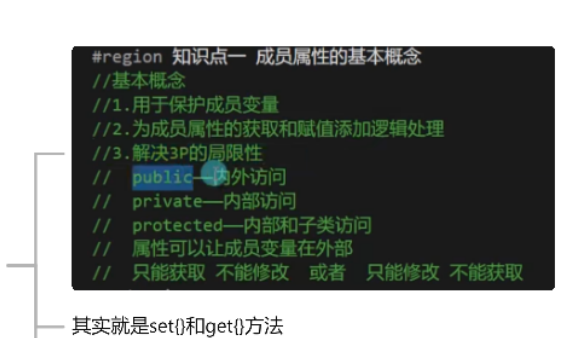
      - 搬迁对象压缩堆和修改引用地址
        - 其实就是把1代中间断的那些对象压缩的紧凑点,变成连续内存 —— 链表变成顺序表是吧
        - 此时还需要修改指针指向的地址
      - 有引用的就放到1代内存中,没引用的当垃圾回收内存
      - 有新对象的时候,如果0代内存满了,就检查哪个是没有引用的 —— (也就是堆中的内存没有对应栈中的指针)
    - 1代的回收 —— 一直移动,等到1代内存也满了的时候,就移动到2代内存中
      - 0,1代的回收速率也要比2代要迅快很多
      - 因为0,1代需要经常释放?
    - 大内存一般放在2代中 —— 因为0,1代经常释放,计算大内存就很消耗时间
  - 手动执行GC
    - 要是每次都满了才释放,玩起来容易突然卡的批爆



```
static void Main(string[] args)
{
    Student student = new Student();

    //一句实现手动释放
    GC.Collect();
}
```

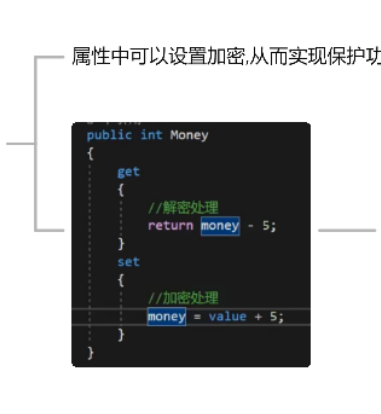
成员属性

- 作用



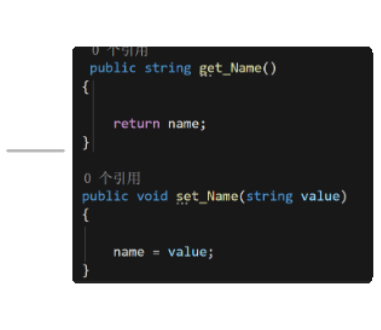
```
#region 知识点一 成员属性的基本概念
//基本概念
//1.用于保护成员变量
//2.为成员属性的获取和赋值添加逻辑处理
//3.解决3P的局限性
//  public—内外访问
//  private—内部访问
//  protected—内部和子类访问
//  属性可以让成员变量在外部
//  只能获取 不能修改  或者  只能修改 不能获取
```

  - 属性中可以设置加密,从而实现保护功能

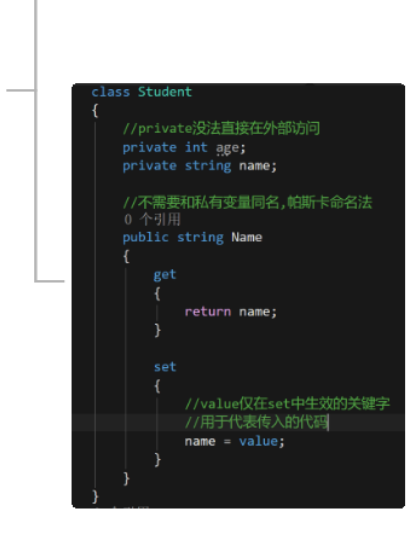


```
public int Money
{
    get
    {
        //解密处理
        return money - 5;
    }
    set
    {
        //加密处理
        money = value + 5;
    }
}
```

    - 读写操作得到的数据不同,防止CE查内存
  - 其实就是set()和get()方法
  - 直白的说就是用来访问内部变量的工具
- 声明
  - 如果get和set方法没有语法糖,完整版

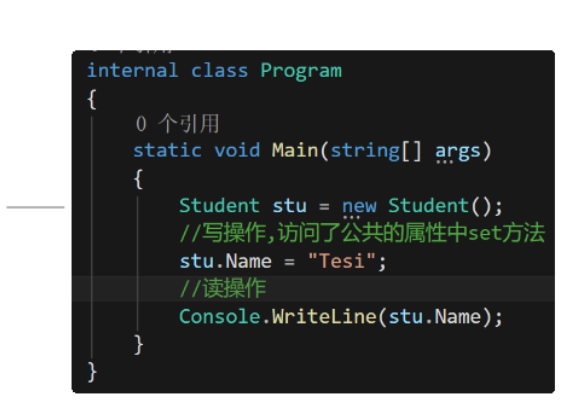


```
public string get_Name()
{
    return name;
}
0 个引用
public void set_Name(string value)
{
    name = value;
}
```



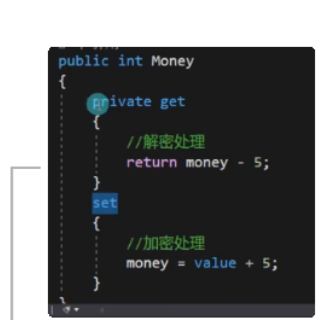
```
class Student
{
    //private没法直接在外部访问
    private int age;
    private string name;

    //不需要和私有变量同名,帕斯卡命名法
    0 个引用
    public string Name
    {
        get
        {
            return name;
        }

        set
        {
            //value仅在set中生效的关键字
            //用于代表传入的代码
            name = value;
        }
    }
}
```

  - value以及name都要和函数返回值(这里是string)相同
- 使用

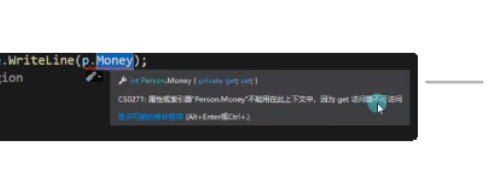


```
internal class Program
{
    0 个引用
    static void Main(string[] args)
    {
        Student stu = new Student();
        //写操作,访问了公共的属性中set方法
        stu.Name = "Tesi";
        //读操作
        Console.WriteLine(stu.Name);
    }
}
```

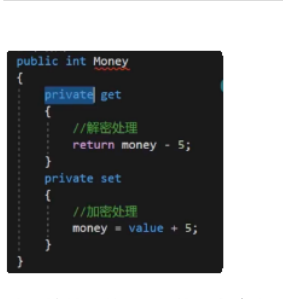
  - 访问get还是set,直接通过你是写还是读来操作,编译器来判断 —— 一种语法糖,所以不用纠结了
- 解决3P的局限性
  - 比如我想在外部访问private这种
  - 通过在get,set前加入修饰符



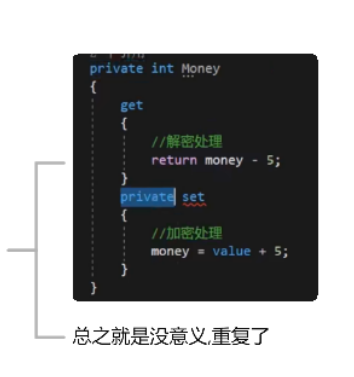
    - 这样外部就只能获取不能修改



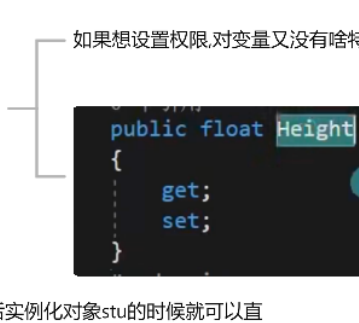
      - 此时访问就会报错



    - 这么两外部public就没有意义了
    - 但是不能两个都是私有,会报错
    - 外面里面都私有,其对接报错



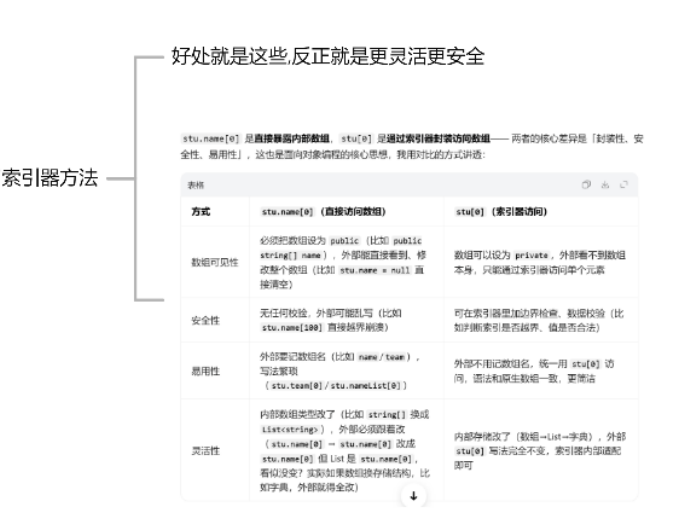
      - 总之就是没意义,是蠢了
  - 可以直接get和set只有一个
    - 同样有效读写,只可读
    - 此时就别多余加个private修饰符了,同样报错
    - 一般也是只会有仅get的情况 —— 毕竟不能得多少有点缘叼了
  - 自动属性(更简单的写法)
    - 如果想设置权限,对变量又没有啥特殊处理

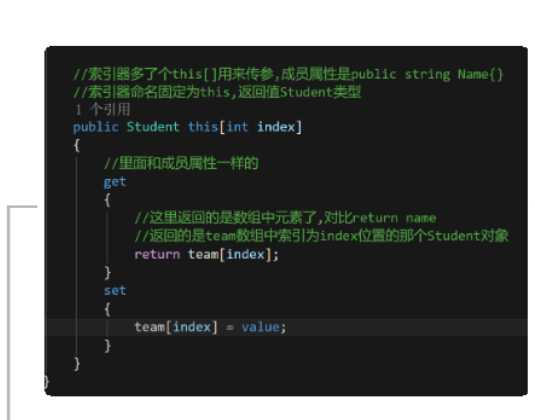


```
public float Height
{
    get;
    set;
}
```

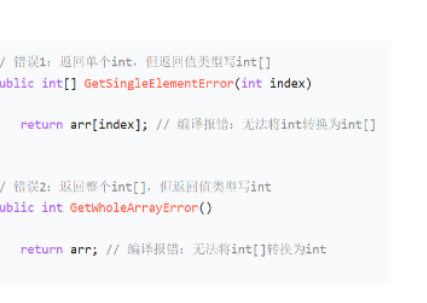
索引器

- 一句话讲明白
  - 成员变量里面设置了一个name数组,之前实例化对象stu的时候是可以直接用 stu访问 —— 好处就是更安全
  - 其实就是是用stu.name[0],只是这个包裹在了索引器中

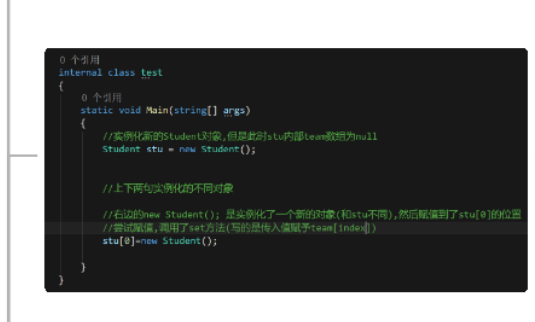




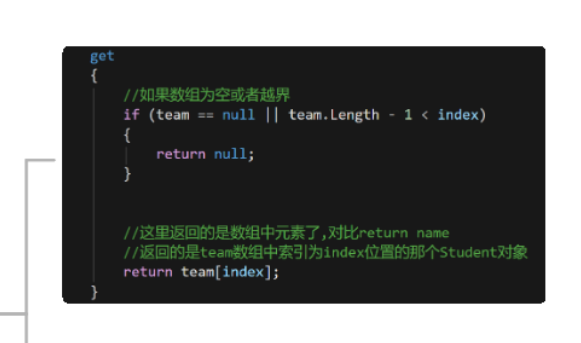
- 函数的返回值和return
  - 返回单个值就是int,返回数组就是int[]不增乱搭配

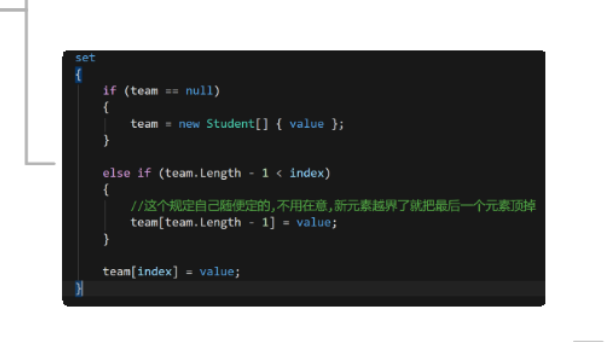


  - 索引器名和固定为this,返回值Student类型
    - 因为这里是return team[index],返回单个值
    - 所以不写return Student[]
- 语法

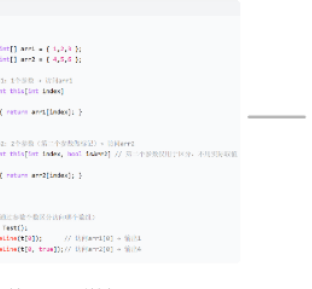


  - stu[0] = stu也是写,不会报错
    - 但是team归我的自己,没有什么用处
    - 相当于自己访问自己的对象里
  - 索引器中写逻辑 —— 访问数组元素前要(小心元素为空)数组越界

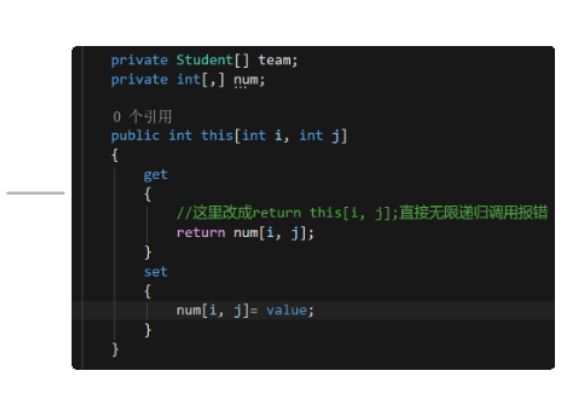




  - 索引器重载
    - 多个数组的时候肯定不能只有一个索引器可用
      - 但是如果有两个int[]数组,直接这种会定义法么重载

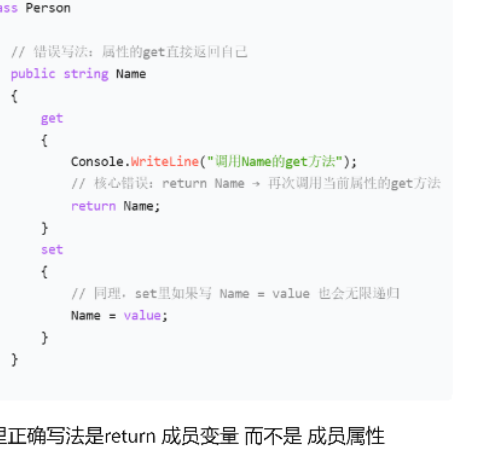


        - 好吧确很乱,加个参数将来区分不就行了 比如bool



```
    {
        //这里改成return this[i, j];直接无限递归调用报错
        return num[i, j];
    }
    set
    {
        num[i, j]= value;
    }
}
```

    - 输入文字
      - 从而无脑进行
      - 但是你访问return this[0,0],就直接回到了调用索引器中的get方法 —— 这里return num[i,j]只是调用成员变量
      - 此时索引访问对象中的索引器get,并且返回num[0,0]
      - 然后我们实例化对象,并且int result = myArray[0, 0];
    - 成员属性中同样可能有无脑调用的情境

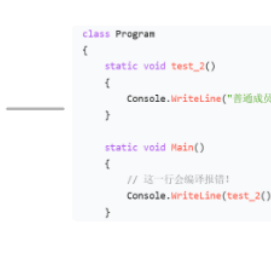


      - 这里正确写法是return 成员变量 而不是 成员属性
    - 如果是return this[i, j];导致无限递归

封装

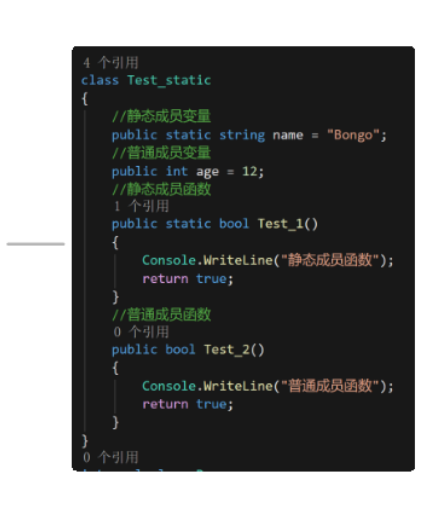
静态成员

- 直接用类名点出使用
  - F12即可查看定义
  - Console类中所有的成员变量都是public static XX{get;set;}
- Console.WriteLine("你好");
  - 关于控制台打印函数



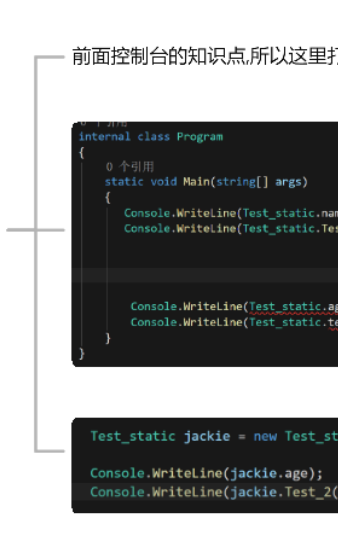
```
class Program
{
    static void test_2()
    {
        Console.WriteLine("普通成员函数");
    }

    static void Main()
    {
        // 这一行会编译报错！
        Console.WriteLine(test_2());
    }
}
```

    - 直接这样只会报错;因为Console.WriteLine() 需要接收一个"有值的参数" —— 而test_2的返回值是void
    - 如果改成返回值是int,然后Console.WriteLine(test_2(99))
      - 会先执行函数中的打印语句+打印99
      - 因为需要先执行函数中的代码,然后return 返回值
- 静态和普通的区别



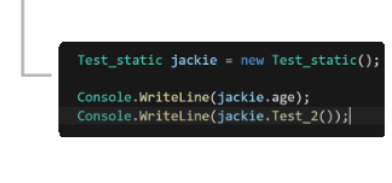
```
4 个引用
class Test_static
{
    //静态成员变量
    public static string name = "Bongo";
    //普通成员变量
    public int age = 12;
    //静态成员函数
    1 个引用
    public static bool Test_1()
    {
        Console.WriteLine("静态成员函数");
        return true;
    }
    //普通成员函数
    0 个引用
    public bool Test_2()
    {
        Console.WriteLine("普通成员函数");
        return true;
    }
}
```

  - 前面控制台的知识点,所以这里打印函数都是bool返回值



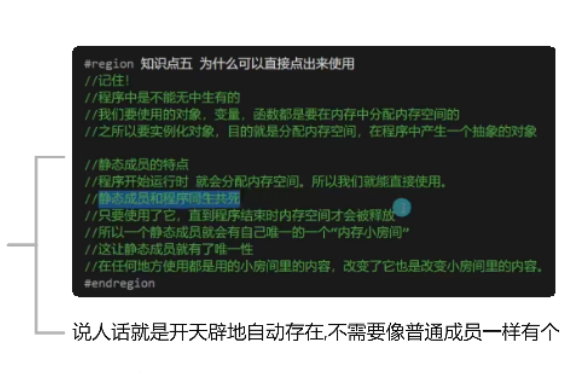
```
internal class Program
{
    0 个引用
    static void Main(string[] args)
    {
        Console.WriteLine(Test_static.name);
        Console.WriteLine(Test_static.Test_1());


        Console.WriteLine(Test_static.age);
        Console.WriteLine(Test_static.test_2());
    }
}
```

  - 静态变量和成员,可以直接类名.成员
  - 注意普通成员类无法,而不是类对象名 —— 就比如Student和jackie



```
Test_static jackie = new Test_static();

Console.WriteLine(jackie.age);
Console.WriteLine(jackie.Test_2());
```

  - 普通的就要先实例化对象 —— 然后对象名.成员
- 静态成员不用实例化的原理

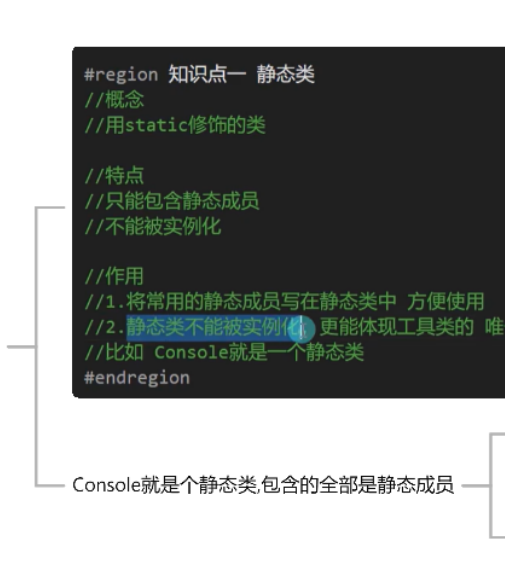


```
#region 知识点五 为什么可以直接点出来使用
//记住!
//程序中是不能无中生有的
//我们要使用的对象, 变量, 函数都是要在内存中分配内存空间的
//之所以要实例化对象, 目的就是分配内存空间, 在程序中产生一个抽象的对象

//静态成员的特点
//程序开始运行时 就会分配内存空间。所以我们就能直接使用。
//静态成员和程序同生共死
//只要使用了它, 直到程序结束时内存空间才会被释放
//所以一个静态成员就会有自己唯一的一个"内存小房间"
//这让静态成员就有了唯一性
//在任何地方使用都是用的小房间里的内容, 改变了它也是改变小房间里的内容。
#endregion
```

  - 普通成员,实例化后存储在堆区中 —— 程序启动后分配，退出释放 —— 和程序是相同的生命周期
  - 静态成员,程序启动后就分配在静态存储区中 —— C#区不依赖类,常量,静态存储
    - 静态成员具有唯一性 —— 就像引用一样,你编辑的始终是那一块区域,而不是搞复制品
  - 说人话就是开天辟地自动存在,不需要像普通成员一样有个出生的过程
- 非静态函数也可以调用静态成员
  - 因为静态成员寿命比较长嘛
  - 静态函数默认不能直接调用非静态成员，但可以「先创建类的实例对象」，再通过对象调用非静态成员。
- 静态成员的作用
  - 声明 常用唯一变量/便于别人获取的对象
  - 但是占用内存且不会释放,不要乱用
- Const常量和Static成员的区别
  - const 是「编译期常量」，静态成员是「运行期变量」
    - 编译期：把你写的「人类能看懂的代码」，... 只做 "翻译和检查"，不执行代码...
    - 运行期：双击 EXE / 启动程序，让... 段才是 "真正干活" 的阶段。
  - const值固定不可改、存储在常量区
    - static可改、存储在静态存储区
    - 而且const常量必须初始化 —— const还必须加上修饰符,static成员就没这种要求
  - 都能通过「类名.成员」直接访问，无需 new 实例对象
    - public const int MAX_AGE = 100;
    - 就可以 Console.WriteLine(Test_static.MAX_AGE); // const常量：100
  - 同样有全局唯一性

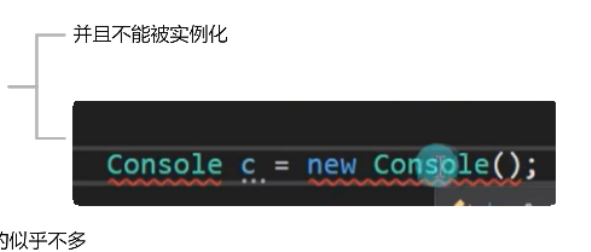
静态类和静态构造函数

- 静态类



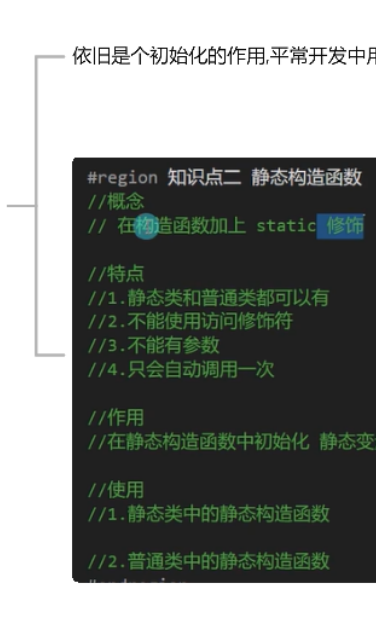
```
#region 知识点一 静态类
//概念
//用static修饰的类

//特点
//只能包含静态成员
//不能被实例化

//作用
//1.将常用的静态成员写在静态类中 方便使用
//2.静态类不能被实例化, 更能体现工具类的 唯一性
//比如 Console就是一个静态类
#endregion
```

  - Console就是个静态类,包含的全部是静态成员
    - 并且不能被实例化



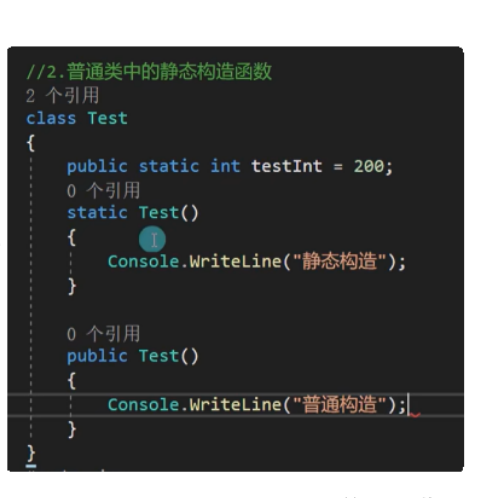
```
Console c = new Console();
```

- 静态构造函数
  - 依旧是个初始化的作用,平常开发中用的可能不多



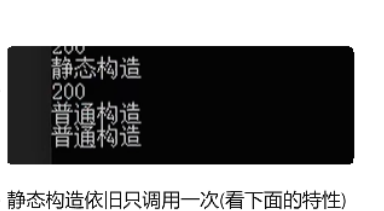
```
#region 知识点二 静态构造函数
//概念
//在构造函数加上 static修饰

//特点
//1.静态类和普通类都可以有
//2.不能使用访问修饰符
//3.不能有参数
//4.只会自动调用一次

//作用
//在静态构造函数中初始化 静态变量

//使用
//1.静态类中的静态构造函数

//2.普通类中的静态构造函数
```

  - 静态类和普通类都可以有 —— 普通类中的静态构造函数



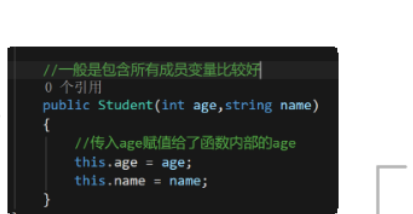
```
//2.普通类中的静态构造函数
0 个引用
class Test
{
    public static int testInt = 200;
    0 个引用
    static Test()
    {
        Console.WriteLine("静态构造");
    }

    0 个引用
    public Test()
    {
        Console.WriteLine("普通构造");
    }
}
```

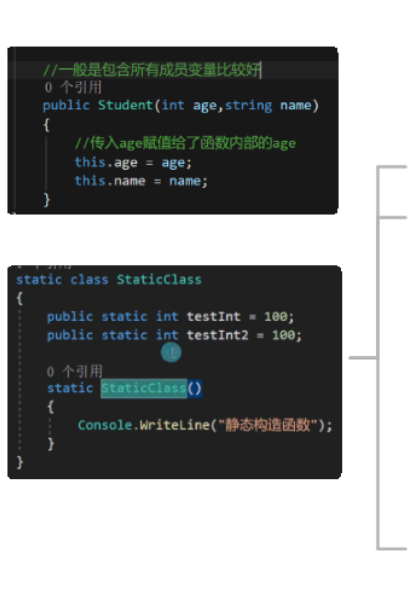
    - 两个Test()参数一样,修饰符不同
      - 并不是重载
      - 这里只是两种不同的构造函数
    - 普通构造函数会调用多次



```
静态构造
200
普通构造
普通构造
```

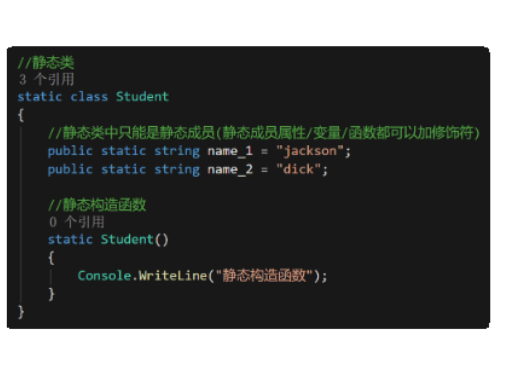
      - 静态构造函数只调用一次(看下面的特性)
  - 对比普通的构造函数

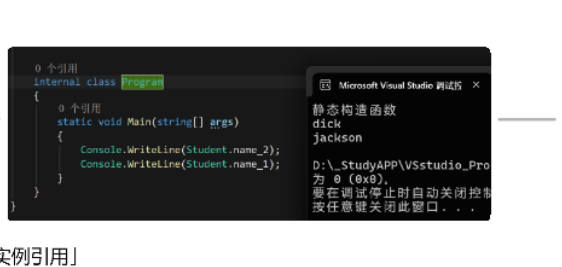




```
static class StaticClass
{
    public static int testInt = 100;
    public static int testInt2 = 100;
    0 个引用
    static StaticClass()
    {
        Console.WriteLine("静态构造函数");
    }
}
```

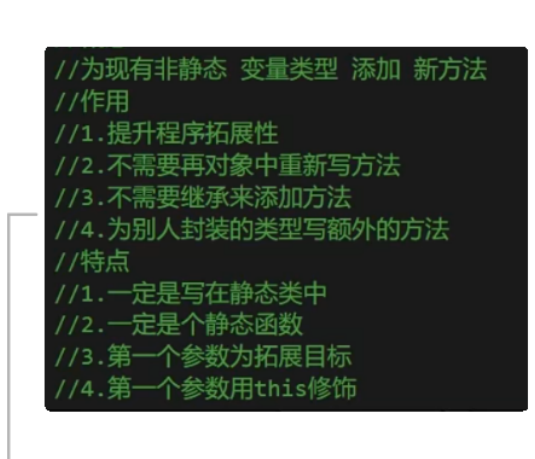
    - 不能有参数
    - 不能有访问修饰符 —— C# 中，绝对不存在 public static 构造函数 这种写法
    - 只调用一次





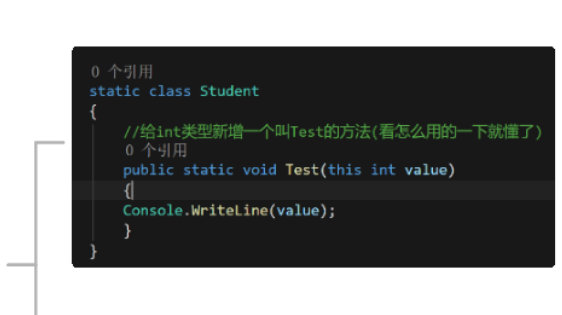
      - 这里访问了两次成员,构造函数却只调用了一次

拓展方法

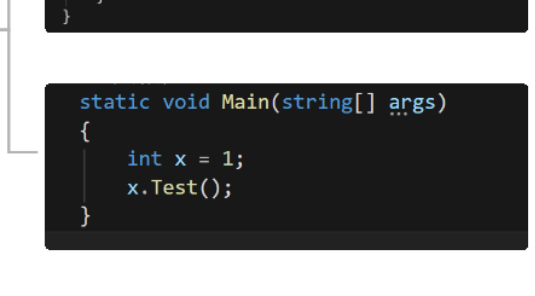


```
//为现有非静态 变量类型 添加 新方法
//作用
//1.提升程序拓展性
//2.不需要再对象中重新写方法
//3.不需要继承来添加方法
//4.为别人封装的类型写额外的方法
//特点
//1.一定是写在静态类中
//2.一定是个静态函数
//3.第一个参数为拓展目标
//4.第一个参数用this修饰
```

- 一定是静态类中的静态函数
- 声明和使用

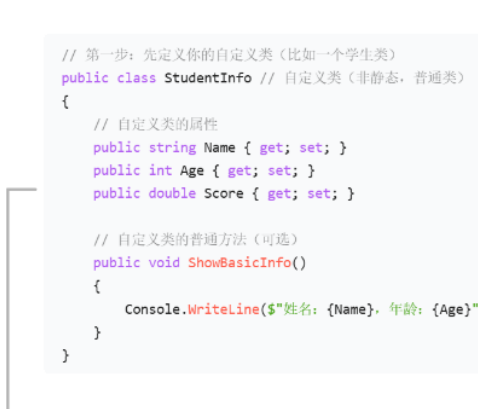


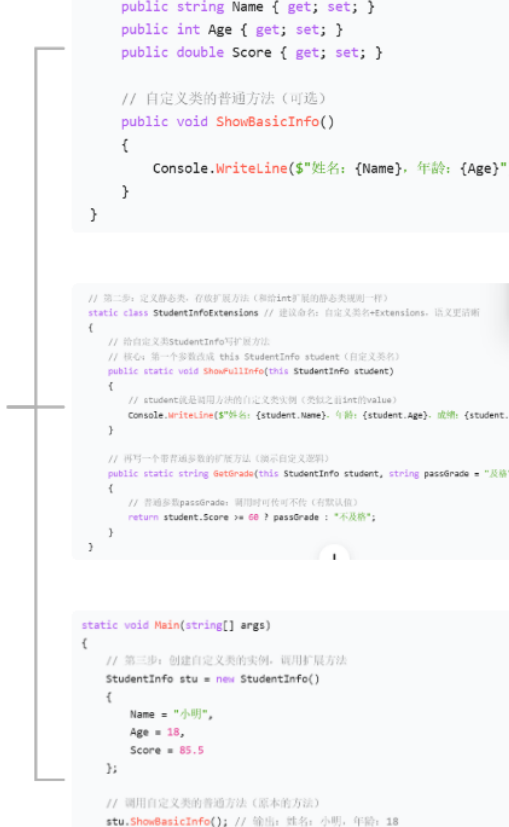
  - 又又又你麻麻是语法糖,老子要吐了
  - 这里的this和索引器没有关系 —— 扩展方法的 this 是「扩展标识」，索引器的 this 是「当前实例引用」
  - 拓展方法Test第一个参数用来声明拓展的类型
    - 剩下的参数就是正常的函数传参
    - 所以必须有至少一个参数

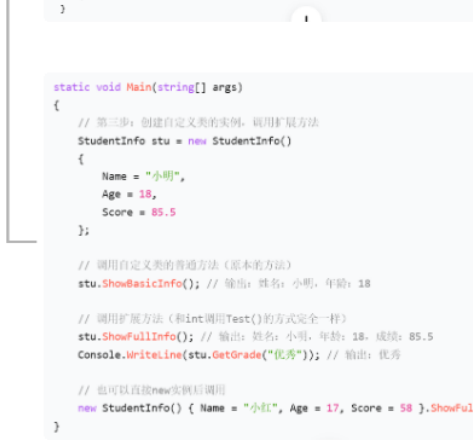


```
static void Main(string[] args)
{
    int x = 1;
    x.Test();
}
```

  - 就是相当于实例化了int类型对象x,然后调用x的Test()方法
  - 打印出来的就是x的初始值value —— 就是正常的x作为参数传入value
- 为自定义的类型拓展方法

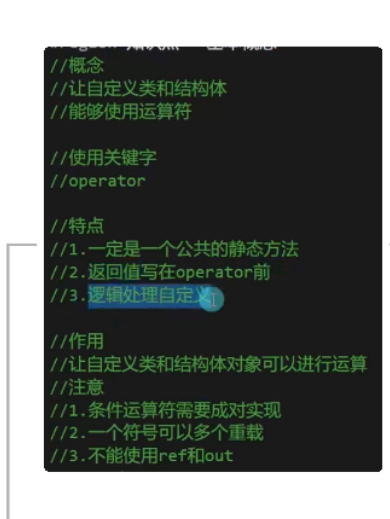




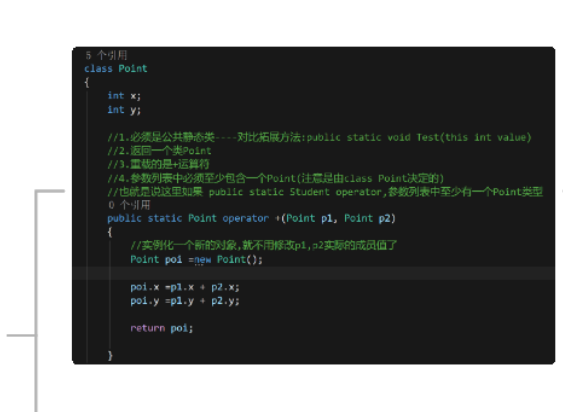


  - 注意不管你的自定义类是普通还是静态类 —— 都必须专门用个另外的静态类存储拓展方法
  - 参数列表this StudentInfo student和this int value一样的
- 如果拓展方法和类型原有方法重名(也没有重载),不会报错但是失效
  - 就像你给Console类又拓展了个writeLine,那肯定还是执行正常的
  - Console.WriteLine(value);
  - 而不是你拓展的那个方法

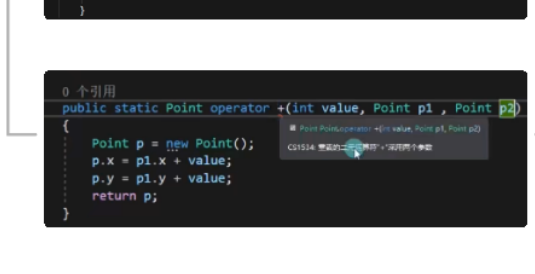
运算符重载



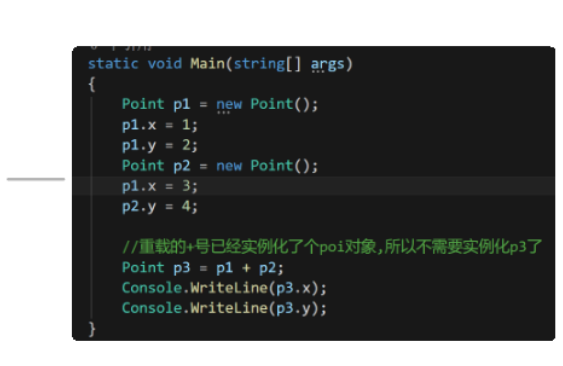
- 关键字operator
- 让自定义类和结构体能使用运算符 —— 比如让Girl和Dog两个结构体能进行加法
- 声明



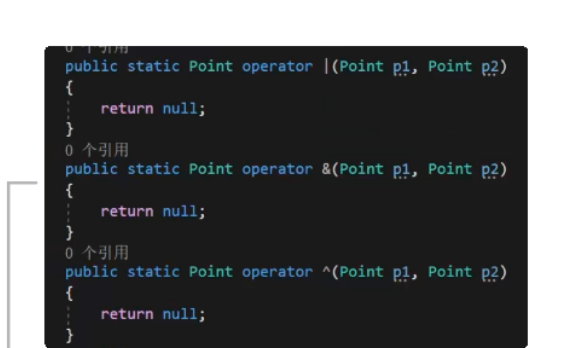
  - 参数列表含有Point和类的类型Point相同



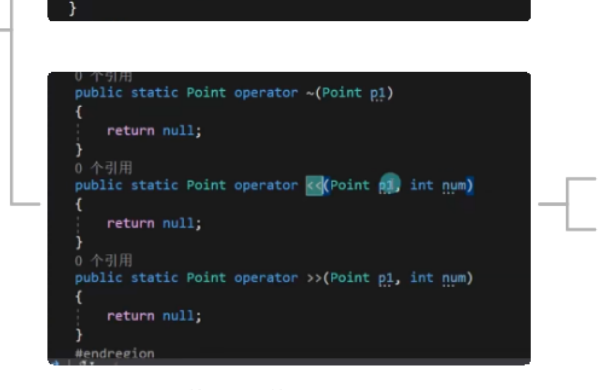
  - 二元运算符重载了仍然只能有两个参数
  - 不能写三个参数
- 输入文字
- 使用



- 可重载的运算符
  - 算数运算符
    - 自增减预算符++/-- —— 这里就是只有一个参数了
    - 加减乘除
  - 逻辑运算符
    - 逻辑非!
    - 「逻辑或（||）」和「逻辑与（&&）」特殊,不能重载
      - 不能重载!
      - 很神秘呢
  - 位运算符

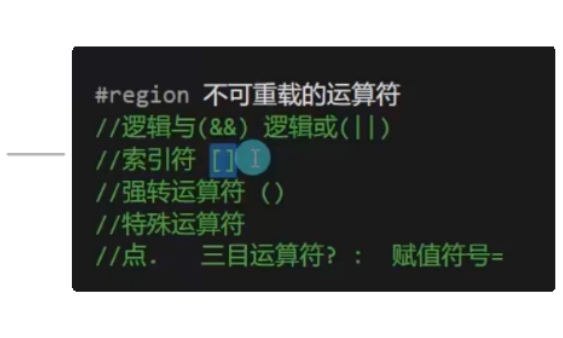


```
public static Point operator |(Point p1, Point p2)
{
    return null;
}
0 个引用
public static Point operator &(Point p1, Point p2)
{
    return null;
}
0 个引用
public static Point operator ^(Point p1, Point p2)
{
    return null;
}
```



```
public static Point operator ~(Point p1)
{
    return null;
}
0 个引用
public static Point operator <<(Point p1, int num)
{
    return null;
}
0 个引用
public static Point operator >>(Point p1, int num)
{
    return null;
}
```

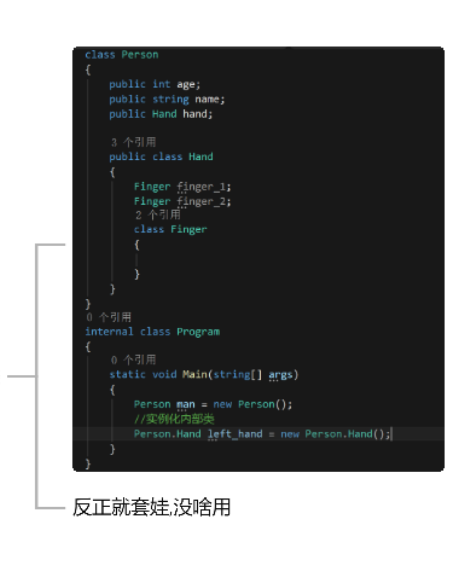
    - 左右移的就是两个参数
    - 因为需要指定移动的位数
  - 条件运算符
    - 大于小于,大于等于,小于等于...
    - 条件运算符需要成对实现
      - 重载了>=就要一起重载<=
      - 重载了==就要重载!=
  - 输入文字
- 不能重载的运算符



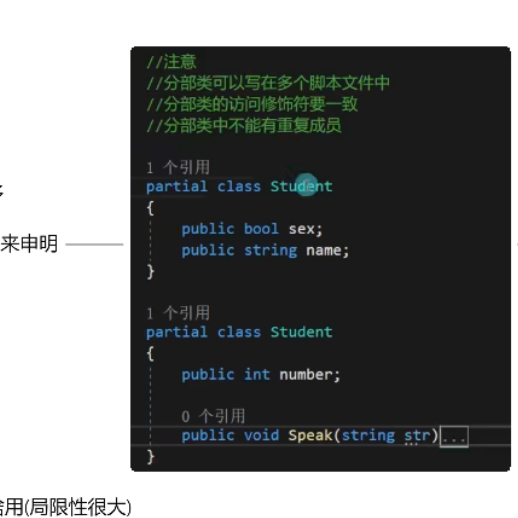
```
#region 不可重载的运算符
//逻辑与(&&) 逻辑或(||)
//索引符 []
//强转运算符 ()
//特殊运算符
//点.  三目运算符?:  赋值符号=
```

内部类和分部类

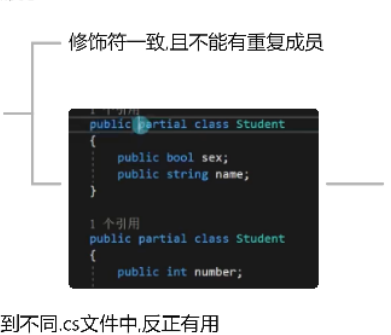
- 内部类
  - 了解即可,不是很重要



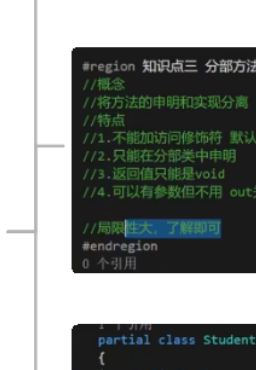
  - 类中再申明一个类 —— 反正就套娃,没啥用
- 分部类
  - 这个有点用,但不多
  - 一个类分成几个类来申明



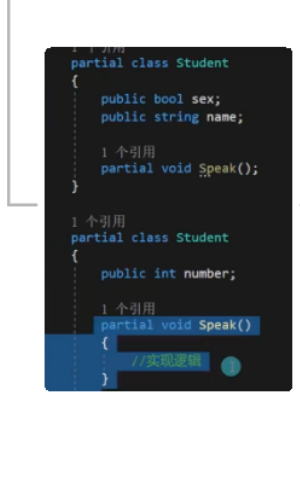
    - 就是分开写成员,没了
    - 前面加个partial
      - 修饰符一致,且不能有重复成员



        - 这个加了public另一个也要加
      - 但它一个类分散到不同.cs文件中,反正有用
- 分部方法
  - 了解下就行,没啥用(局限性太大)



  - 输入文字



  - 上面的声明,下面的实现就分开了

by 狗大王 由 幕布 发布